

Manual JavaScript 30 Días - Bootcamp Completo

Introducción

Este manual está diseñado para llevarte desde fundamentos hasta la creación de aplicaciones CRUD con JavaScript puro en 30 días. Cada día incluye teoría, ejemplos, buenas prácticas, ejercicios y retos.

Cómo usar este manual

- Estudia 1 día por jornada.
 - Toma apuntes propios.
 - Escribe el código a mano.
 - Modifica ejemplos.
 - Completa los retos.
-

Semana 1 - Fundamentos

Día 1: Variables y tipos de datos

Qué son las variables

Una variable es un espacio con nombre donde guardas información para reutilizarla después. En JavaScript se usan principalmente `let` y `const`. `let` permite reasignar valores; `const` mantiene la referencia fija. Elegir correctamente entre ambas mejora la claridad del código y evita errores.

En proyectos reales, conviene usar `const` por defecto y `let` solo cuando el valor realmente cambiará. Esto hace más predecible el comportamiento del programa y facilita el mantenimiento.

```
let nombre = "Ana";  
const edad = 24;
```

Tipos de datos principales

JavaScript maneja tipos primitivos como string, number, boolean, null y undefined. También maneja estructuras como objects y arrays. Comprender los tipos evita errores al comparar, sumar o transformar datos.

Usa `typeof` para inspeccionar valores cuando depuras código.

```
typeof "Hola"; // string
typeof 10; // number
typeof true; // boolean
```

Ejercicios

1. Crea 10 variables de distintos tipos.
2. Muestra cada tipo con `typeof`.
3. Cambia una variable declarada con `let`.

Día 2: Operadores y condicionales

Operadores

Los operadores permiten realizar cálculos y comparaciones. Los aritméticos incluyen suma, resta, multiplicación, división y módulo. Los operadores de comparación devuelven true o false.

Es recomendable usar `===` en lugar de `==`, porque compara valor y tipo, evitando conversiones implícitas inesperadas.

```
5 + 3;
10 % 3;
5 === "5"; // false
```

Condicionales

Los condicionales permiten tomar decisiones. Si una condición se cumple, ejecutas un bloque; si no, otro. Son la base de validaciones, flujos de formularios y reglas de negocio.

```
if (edad >= 18) {
  console.log("Puede ingresar");
} else {
  console.log("Acceso denegado");
}
```

Ejercicios

1. Verifica mayoría de edad.
 2. Calcula si un número es par o impar.
 3. Evalúa nota aprobada o reprobada.
-

Día 3: Bucles

Qué son los bucles

Los bucles repiten instrucciones sin duplicar código. `for` es ideal cuando conoces la cantidad de repeticiones; `while` cuando depende de una condición dinámica.

Dominar bucles es clave para recorrer arrays, renderizar listas en pantalla y procesar datos masivos.

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Buenas prácticas

Evita bucles infinitos. Verifica siempre que la condición eventualmente cambie. Usa nombres descriptivos para índices cuando tenga sentido.

Ejercicios

1. Imprime números del 1 al 100.
 2. Tabla del 7.
 3. Solo números pares.
-

Día 4: Funciones

Qué son las funciones

Una función agrupa instrucciones reutilizables. Permite dividir problemas grandes en piezas pequeñas. Esto mejora organización, pruebas y mantenimiento.

Puedes usar funciones tradicionales o arrow functions. Ambas son útiles, aunque tienen diferencias en `this` y comportamiento interno.

```
function sumar(a, b) {  
  return a + b;  
}  
  
const restar = (a, b) => a - b;
```

Ejercicios

1. Área de rectángulo.
 2. Convertir Celsius a Fahrenheit.
 3. Función saludar(nombre).
-

Día 5: Arrays

Qué es un array

Un array almacena múltiples valores en orden. Es ideal para listas: tareas, productos, usuarios, mensajes. Cada elemento tiene una posición iniciando en 0.

Aprender arrays es fundamental porque gran parte del trabajo web consiste en mostrar colecciones de datos.

```
const tareas = ["Estudiar", "Leer", "Practicar"];
```

Métodos comunes

`push()` agrega al final, `pop()` elimina el último, `shift()` elimina el primero y `unshift()` agrega al inicio.

Ejercicios

1. Crear lista de compras.
 2. Agregar 3 productos.
 3. Eliminar el último.
-

Día 6: Objetos

Qué es un objeto

Un objeto agrupa propiedades relacionadas. Sirve para modelar entidades reales: usuario, producto, pedido, vehículo. Cada propiedad tiene clave y valor.

Pensar en objetos es pensar en datos estructurados, algo central en frontend y backend.

```
const usuario = {  
  nombre: "Luis",  
  edad: 22,  
  activo: true  
};
```

Acceso y modificación

```
usuario.nombre;  
usuario.edad = 23;
```

Ejercicios

1. Crear objeto producto.
 2. Modificar precio.
 3. Mostrar propiedades.
-

Día 7: Mini Proyecto

Calculadora básica

Construye una calculadora con dos inputs y botones para sumar, restar, multiplicar y dividir. Este proyecto combina variables, funciones, eventos y DOM básico.

Objetivo técnico

Separar lógica matemática de manipulación visual es una buena práctica. Crea funciones puras para operaciones y otra capa para interfaz.

Semana 2 - DOM y Eventos

Día 8: Selección del DOM

Qué es el DOM

El DOM (Document Object Model) representa el HTML como una estructura de nodos que JavaScript puede leer y modificar. Gracias al DOM puedes cambiar textos, estilos, atributos y responder a acciones del usuario.

Comprender el DOM transforma páginas estáticas en interfaces dinámicas. Todo botón, formulario o lista interactiva depende de esta relación entre HTML y JavaScript.

```
document.querySelector("#app");  
document.getElementById("titulo");
```

Día 9: Eventos

Los eventos son acciones como clics, escritura, envío de formularios o movimiento del mouse. JavaScript escucha estos sucesos y ejecuta funciones.

```
boton.addEventListener("click", guardarDatos);
```

Usa funciones separadas para mantener orden y reutilización.

Día 10: Crear elementos

Puedes construir nodos nuevos desde JavaScript sin escribir HTML manualmente. Esto se usa en listas dinámicas, tarjetas de productos y tablas.

```
const li = document.createElement("li");
li.textContent = "Nueva tarea";
lista.appendChild(li);
```

Día 11: Modificar elementos

Modificar contenido y clases es común al actualizar estados visuales.

```
titulo.textContent = "Panel actualizado";
caja.classList.add("activo");
```

Día 12: Eliminar elementos

Eliminar nodos es útil para tareas, notificaciones y registros.

```
elemento.remove();
```

Día 13: Formularios

Capturar datos correctamente requiere prevenir recargas por defecto.

```
form.addEventListener("submit", e => {
  e.preventDefault();
});
```

Valida campos antes de procesar datos.

Día 14: Proyecto semanal

Construye una lista de tareas con:

- agregar tareas
 - completar tareas
 - eliminar tareas
 - contador de pendientes
-

Semana 3 - Local Storage y Calidad de Código

Día 15: Local Storage

Local Storage permite guardar datos en el navegador incluso al cerrar la pestaña. Es útil para preferencias, tareas locales y borradores.

```
localStorage.setItem("tema", "dark");
```

Día 16: Leer datos guardados

```
const tema = localStorage.getItem("tema");
```

Siempre verifica si existe información antes de usarla.

Día 17: JSON

Local Storage guarda texto. Para almacenar arrays u objetos debes convertirlos con JSON.

```
JSON.stringify(usuario);  
JSON.parse(texto);
```

Día 18: Persistir colecciones

Guarda listas completas de tareas o productos.

```
localStorage.setItem("tareas", JSON.stringify(tareas));
```

Día 19: Validación de datos

Validar evita errores y mejora experiencia del usuario.

```
if(input.value.trim() === "") {  
  alert("Campo obligatorio");  
}
```

Valida longitud, formato y duplicados cuando aplique.

Día 20: Manejo de errores

Los errores ocurren. Debes capturarlos para evitar fallos silenciosos.

```
try {
  ejecutar();
} catch(error) {
  console.error(error.message);
}
```

Día 21: Proyecto semanal

To-Do persistente:

- guarda tareas
- recupera tareas al cargar
- elimina tareas
- valida campos vacíos

Semana 4 - Fetch API, CRUD y Proyecto Final

Día 22: Fetch GET

Fetch permite comunicarte con servidores y APIs. GET obtiene información.

```
const res = await fetch(url);
const data = await res.json();
```

Comprende que muchas apps modernas viven de datos remotos.

Día 23: POST

POST crea nuevos recursos.

```
fetch(url, {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify(datos)
});
```

Día 24: PUT

PUT actualiza registros completos.

```
method: "PUT"
```


Día 25: DELETE

DELETE elimina registros.

```
method: "DELETE"
```

Día 26: Async / Await

Hace código asíncrono más legible que promesas encadenadas.

```
async function cargar(){  
  const res = await fetch(url);  
  return await res.json();  
}
```

Día 27: Modularizar código

Divide archivos por responsabilidad.

```
app.js  
api.js  
ui.js  
storage.js
```

Esto mejora mantenimiento y escalabilidad.

Día 28: Buenas prácticas profesionales

Usa camelCase:

```
obtenerUsuarios();  
guardarProducto();  
listaTareas;
```

Comentarios útiles explican por qué, no lo obvio.

```
// Reintenta la petición si falla por red
```

Día 29: Proyecto Final - Planeación

Elige uno:

- Inventario

- Agenda
- ToDo Pro

Define:

- datos a guardar
- vistas necesarias
- eventos
- almacenamiento
- integración API

Día 30: Proyecto Final - Entrega

Construye la versión funcional con:

- crear
- leer
- editar
- eliminar
- validaciones
- persistencia
- diseño ordenado

Recursos recomendados

- MDN Web Docs
- JavaScript.info
- freeCodeCamp
- JSONPlaceholder

Consejos finales

1. Escribe código todos los días.
2. Repite proyectos cambiando temática.
3. Lee errores en consola con calma.
4. Refactoriza código antiguo.
5. Publica proyectos en GitHub.

Resultado esperado al terminar

Serás capaz de crear aplicaciones web interactivas con JavaScript puro, manejar datos locales y remotos, estructurar proyectos correctamente y estar listo para aprender React o backend con Node.js.